

How We Built It

Step-by-Step Build Procedure

The complete, ordered engineering playbook used to build SwiftMatch — from an empty monorepo to a live, assessment-first recruitment platform across 22 industries.

Project: SwiftMatch — "Don't search. Get spotted."

Stack: pnpm monorepo · TypeScript · React + Vite · Express 5 · PostgreSQL + Drizzle

Version: 1.0 · **Date:** June 2026 · **Technical Documentation**

Build Roadmap

SwiftMatch was built in 14 sequential phases. Each phase below lists its goal, the actions taken, and the key outcome.

01 Project & Monorepo Foundation	pnpm workspace + TypeScript
02 Database Schema Design	PostgreSQL + Drizzle ORM
03 Contract-First API Spec	OpenAPI + Zod + Orval
04 Backend API Server	Express 5 routes & seeding
05 Frontend Scaffold	React + Vite + Tailwind
06 Applicant Profile Wizard	10-step onboarding
07 Résumé Auto-Fill (AI)	PDF/DOCX → GPT parsing
08 Assessment Engine	5 dimensions × 22 industries
09 Jobs, Applications & Matching	Apply flow + scoring
10 Recruiter Custom Assessments	Per-job question builder
11 Employer Report & Decisioning	Ranked pool + Pass/Fail
12 Authentication & Owner Accounts	JWT + role bypasses
13 Premium Gating	Freemium paywall
14 Supporting Artifacts & Launch	Marketing, video, deploy

Project & Monorepo Foundation

Goal: A clean, type-safe multi-package workspace where each app and library manages its own dependencies.

Steps

1. Initialize a **pnpm workspace monorepo** with `pnpm-workspace.yaml` declaring the `artifacts/*` and `lib/*` package globs.
2. Create a shared `tsconfig.base.json` with strict TypeScript defaults (`composite: true` , incremental builds) and a root `tsconfig.json` solution file referencing all composite libraries.
3. Pin shared dependency versions in the workspace **catalog** so every package resolves the same React, TypeScript, etc.
4. Establish the folder structure: deployable apps in `artifacts/` , shared code in `lib/` , utility scripts in `scripts/` .

```
# Resulting structure
artifacts-monorepo/
├─ artifacts/
│  ├─ api-server/      # Express API
│  └─ swiftmatch/      # React + Vite frontend
├─ lib/
│  ├─ api-spec/        # OpenAPI + Orval config
│  ├─ api-client-react/ # Generated React Query hooks
│  ├─ api-zod/         # Generated Zod schemas
│  └─ db/              # Drizzle schema + connection
├─ pnpm-workspace.yaml
└─ tsconfig.base.json
```

Principle: Leaf apps (`artifacts/*`) never import each other. Anything shared becomes a `lib/*` package — keeping boundaries clean and types portable.

Database Schema Design

Goal: A structured talent graph in PostgreSQL, modeled with Drizzle ORM in the `@workspace/db` library.

Steps

1. Provision a **PostgreSQL** database; expose its connection via the `DATABASE_URL` environment variable.
2. Model the domain as Drizzle schema files, one per concern, under `lib/db/schema/`.
3. Use `drizzle-zod` to derive validation schemas directly from table definitions, keeping DB and API contracts in sync.
4. Push schema changes with `pnpm --filter @workspace/db run push` (no manual SQL migrations).

Core Tables

SCHEMA FILE	HOLDS
<code>applicants.ts</code>	Candidate profiles: personal info, contact, skills, history, references, preferences
<code>assessments.ts</code>	Assessment definitions + <code>assessment_results</code> (with nullable <code>job_id</code> FK)
<code>jobs.ts</code>	Job postings, <code>custom_questions</code> JSONB, and <code>job_applications</code>
<code>courses.ts</code>	Training / course recommendations (the skill-enhancement "bench")

```
# job_applications – the join between candidate and job
(id, applicant_id, job_id, job_title, company,
 industry, status, ke_score,
 custom_score, custom_correct_count, custom_total_count,
 custom_answers JSONB, created_at)
UNIQUE(applicant_id, job_id)
```

Contract-First API Spec

Goal: Define every endpoint once in OpenAPI, then generate type-safe clients and validators — no hand-written API glue.

Steps

1. Author the **OpenAPI specification** in `lib/api-spec/` describing all routes, request bodies, and responses.
2. Configure **Orval** to generate, from that single spec:
 - `@workspace/api-client-react` — React Query hooks for the frontend.
 - `@workspace/api-zod` — Zod schemas for runtime validation on both sides.
3. Run codegen whenever the contract changes:

```
pnpm --filter @workspace/api-spec run codegen
```

Why contract-first: The server validates inputs/outputs with the generated Zod schemas, and the client calls the generated hooks. The compiler catches any drift between frontend and backend before it ships.

Note: the OpenAPI `info.title` drives generated filenames, so it stays fixed once established.

Backend API Server

Goal: An Express 5 + TypeScript server exposing the talent graph, with auto-seeded demo data.

Steps

1. Scaffold the `@workspace/api-server` Express application; bind it to the `PORT` env var and mount everything under the `/api` base path.
2. Implement route handlers grouped by concern (applicants, assessments, jobs, courses, skills, résumé, auth).
3. Validate all inputs and outputs against the generated Zod schemas.
4. Use structured logging (`req.log` in handlers) — never `console.log` in server code.
5. Auto-seed demo content on boot: assessments, 6 sample jobs, 8 courses.

Key Endpoints

ROUTE	PURPOSE
GET <code>/api/healthz</code>	Health check
GET/POST <code>/api/applicants</code>	List / create candidate profiles
GET <code>/api/jobs</code> · <code>/:id</code>	List / fetch postings (answer keys stripped)
POST <code>/api/jobs/:id/apply</code>	Apply to a job (JWT required, 409 if duplicate)
POST <code>/api/resume/parse</code>	Parse uploaded résumé into structured data
GET <code>/api/skills/suggestions?q=</code>	Skill autocomplete

Frontend Scaffold

Goal: A fast, polished React single-page app served at the root path.

Steps

1. Scaffold the `@workspace/swiftmatch` app with **React + Vite**.
2. Add **Tailwind CSS** and **shadcn/ui** for a consistent design system; **Framer Motion** for animation.
3. Configure Vite to read `PORT` and allow proxied hosts (`server.allowedHosts: true`) so the preview renders through Replit's iframe proxy.
4. Wire data fetching through the generated `@workspace/api-client-react` hooks — using base-path-aware URLs, never root-relative ones.
5. Establish the brand: navy primary, orange accent, the "Don't search. Get spotted." identity.

Routing model: A global reverse proxy routes by path — the frontend lives at `/`, the API at `/api`. Each service handles its own full base path; no Vite proxy hacks needed.

Applicant Profile Wizard

Goal: Capture a rich, structured candidate profile through a guided 10-step flow.

The 10 Steps

#	STEP	CAPTURES
1	Personal Info	Name, pronoun, nickname, suffix
2	Contact Info	Addresses, phone (area code), email
3	Skills	API autocomplete, max 5 tags
4	Employment History	Company, position, years, reason for leaving
5	Certificates & Training	Up to 3 entries
6	Character References	Up to 3, with phone + email
7	Social Links	Facebook + LinkedIn
8	Job Preferences	Salary, negotiable toggle, availability date
9	Pre-Assessment	4 categories of baseline questions
10	Introduction Video	Upload

Steps to Build

1. Model each step as a discrete form component with local validation.
2. Persist progress to the applicant record via the generated mutation hooks.
3. Add the résumé upload card above the wizard (see Phase 07) to pre-fill the steps.

Résumé Auto-Fill (AI)

Goal: Remove onboarding friction — let a candidate upload a résumé and have the profile fill itself.

Steps

1. Add a résumé upload card (PDF, .docx/.doc, plain text; up to 10MB) above the 10-step wizard.
2. Implement `POST /api/resume/parse` (multipart, field `resume`).
3. Extract raw text server-side: `pdf-parse` for PDFs, `mammoth` for DOCX.
4. Send the text to **GPT (gpt-5-mini)** via the Replit-managed OpenAI AI Integration — no user API key required.
5. Map the model's structured output to wizard fields: name, contact, address, skills, employment history, certificates, salary, availability, social links.

```
# Managed via env vars — billed to Replit credits
AI_INTEGRATIONS_OPENAI_BASE_URL
AI_INTEGRATIONS_OPENAI_API_KEY
```

Result: A candidate goes from "upload" to a mostly-complete profile in seconds, then reviews and refines — dramatically improving completion rates.

Assessment Engine

Goal: Score every candidate objectively across five dimensions and 22 industries — fairly and un-gameably.

The Five Dimensions

KNOWLEDGE & EXPERTISE

Industry competency; difficulty-weighted, role-prioritized banks.

PERSONALITY & WORK STYLE

DOPE Bird / MBTI routing by role & seniority.

CULTURAL FIT

Workplace-norm alignment, per industry.

CRITICAL THINKING

Scenario-based reasoning.

AI READINESS

50-question industry-scoped bank measuring practical AI-tool fluency — the differentiator.

Steps

1. Build the question banks in `industry-questions.ts` with `INDUSTRY_QUESTIONS`, `INDUSTRY_ROLES`, and an `ANSWER_KEY` per industry.
2. Implement `pickQuiz` to randomize 10 questions per session (3 easy, 4 medium, 3 hard) so no two candidates get the same test.
3. Grade **server-side** only — the client never sees answer keys.
4. Enforce a **1-month retake cooldown** (bypassed for the owner account during testing).
5. Normalize raw industry strings to canonical quiz keys via `normalizeIndustry()`.
6. Scale coverage to **22 industries** — including adding Virtual Assistance end-to-end (question bank, routing, role maps, slug maps).

Jobs, Applications & Matching

Goal: Let candidates apply to jobs and auto-trigger the right assessment, while linking results back to each application.

Steps

1. Render job cards and a detail panel with an **Apply** button (hidden for employer sessions).
2. On apply: if JWT valid → `POST /api/jobs/:id/apply` → redirect to the assessment pre-loaded with the job's industry/role/company; if no JWT → redirect to signup.
3. Create a `job_applications` row (unique per applicant+job; 409 on duplicates).
4. On K&E submit, pass the `jobId` so the score updates `job_applications.ke_score` and sets `status = "assessed"`.
5. Show a job-context banner in the quiz and auto-start the assessment from URL params or from the profile's target industry/role.

Outcome: Applying and getting measured become a single, seamless flow — every application carries an objective score.

Recruiter Custom Assessments

Goal: Let recruiters attach their own scored questions to a specific job and grade them automatically.

Steps

1. Add a `custom_questions` JSONB column to `jobs`; each question = `{ id, prompt, type, options?, correctAnswers[] }`.
2. Build the recruiter question builder in the Jobs editor — multiple-choice (checkbox for correct option) and text (pipe-separated accepted answers). Use functional `useState` to avoid stale-closure drops on rapid edits.
3. Gate editing with `canEditJob` = admin OR employer-owner.
4. **Strip answer keys** from all public reads (`stripJobAnswerKeys`); expose sanitized questions via `GET /api/jobs/:id/custom-assessment`.
5. Grade on submit (`POST .../custom-assessment/submit`): case-insensitive trimmed equality; persist atomically with `INSERT ... ON CONFLICT ... DO UPDATE` when a profile exists.

Security principle: Correct answers never leave the server for public consumers. The grading boundary lives entirely server-side.

Employer Report & Decisioning

Goal: Give employers a ranked, comparable candidate pool and a way to act on it.

Steps

1. Build the **Candidate Analytics Report**: a ranked pool scored across all five dimensions, with radar comparison (top candidate vs. pool average).
2. Add a **job-fit matching** view ranking candidates by compatibility (industry, role, level, assessment factors).
3. Render per-candidate **Pass / Fail decision radios** in both tables — emerald Pass, rose Fail — isolated from the row's "View Report" click.
4. Hold decisions in a state map keyed by candidate id (UI-first), ready to wire to a backend column and notifications.

Planned Next (decisioning pipeline)

- Persist decisions to a DB column via a new endpoint.
- On **Pass** → email the candidate a hiring-process invite.
- On **Fail** → respectful thank-you email plus training / alternative-industry suggestions.

Authentication & Owner Accounts

Goal: Identify candidates vs. employers vs. admins, and enable safe testing bypasses.

Steps

1. Implement signup/login issuing a **JWT**; protect apply, submit, and admin routes with it.
2. Maintain owner/admin emails in two synced places: backend `DEFAULT_OWNER_EMAILS` (extendable via `OWNER_EMAILS` env var) and frontend `owner.ts` (`isOwnerEmail()`).
3. Add owner bypasses for testing: skip email confirmation on signup, and treat owner accounts as premium.
4. Tie bypasses to the authenticated identity (`useAuth()` + `isOwnerEmail()`), not device-local flags.

Deferred: Email delivery (confirmation + reset). The email lib logs the link to the server console until an SMTP/email integration is connected — a known, documented gap.

Premium Gating

Goal: Monetize the full analytics report behind a premium tier.

Steps

1. Compute premium status on the Results page: `isPremium = localStorage.sm_subscription === "active" || isOwnerEmail(user?.email) .`
2. Wrap the full report in a `LockOverlay` — blurred preview + subscribe upsell for free users.
3. Owner accounts see the full report with no lock, for demos and testing.

Known limitation & next build: the gate is currently client-side (bypassable). Moving it server-side — so the API itself withholds premium data from unentitled users — is the priority hardening step before charging real customers.

Supporting Artifacts & Launch

Goal: Surround the product with everything needed to demo, market, and ship it.

Steps

1. Build a **buildathon marketing site** artifact (brand/positioning showcase) at its own path.
2. Produce a **2-minute product walkthrough video** artifact.
3. Maintain a **mockup-sandbox** artifact for isolated component previews on the canvas.
4. Verify each artifact with `pnpm --filter @workspace/<slug> run typecheck` (not `build`, which needs workflow env vars).
5. Run each app via its **workflow**; confirm the preview renders through the proxy.
6. Review major features with an architect pass; fix severe issues before shipping.
7. **Deploy** — the platform handles build, hosting, TLS, and health checks.

Standard Verification Loop (used throughout)

COMMAND	CHECKS
<code>pnpm run typecheck</code>	Full workspace type safety (libs then apps)
<code>pnpm --filter @workspace/api-spec run codegen</code>	Regenerate client/Zod after contract changes
<code>pnpm --filter @workspace/db run push</code>	Apply schema changes to the database

From empty repo to live platform — Don't search. Get spotted.

Technical build documentation for SwiftMatch. Reflects the architecture and procedure as of June 2026. © 2026 SwiftMatch.